# Unit – 1

# Introduction to Software and Software Engineering

## Outline

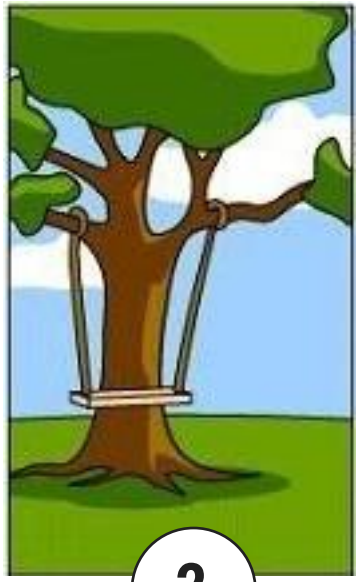
- Software, Characteristics of Software, Software Application Domains
- Software Engineering, Software Engineering Layered Approach
- Software Process, Process Framework Activities , Umbrella Activities
- Software Myths
  - Management Myth
  - Customer Myth
  - Practitioner's/Developer Myth)

# Why to Study Software Engineering?

## Software Development Life Cycle without Software Engineering

1

How the Customer Explains Requirement

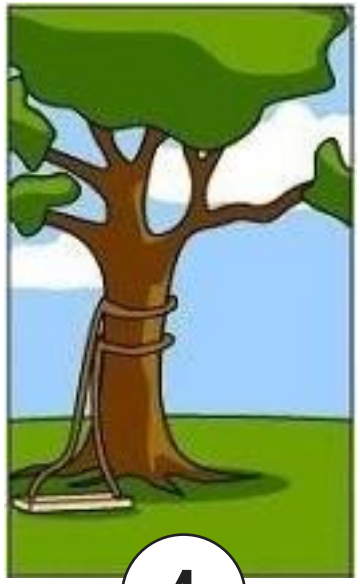


2

How the Project Leader understand it

3

How the System Analyst design it



4

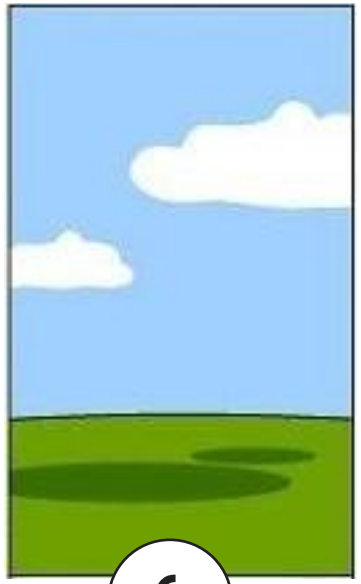
How the Programmer Works on it

5

How the Business Consultant describe it

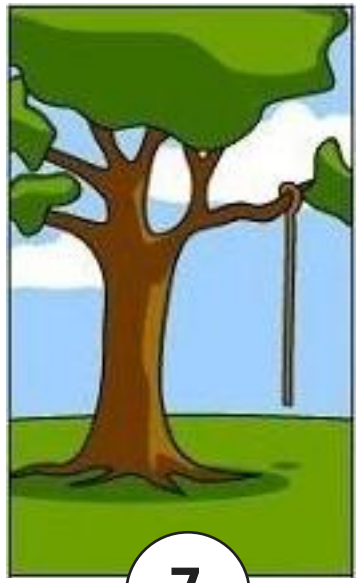
# Why to Study Software Engineering?

## Software Development Life Cycle **without** Software Engineering



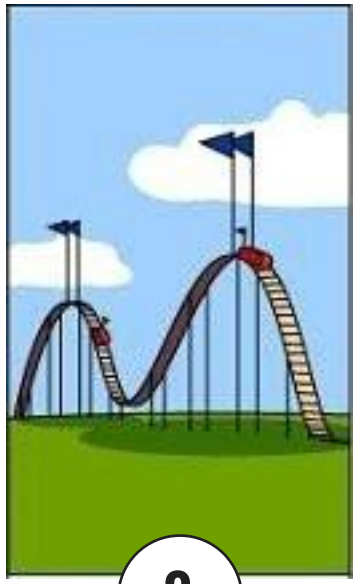
6

How the Project documented



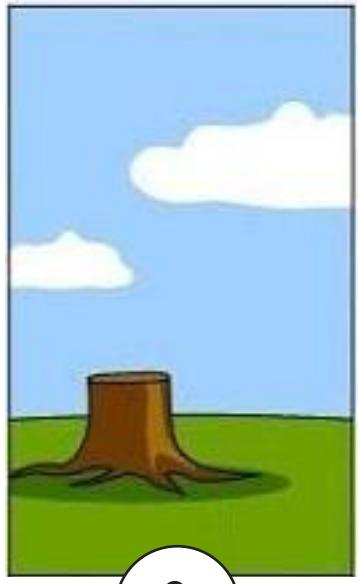
7

What Operations Installed



8

How the Customer billed



9

How it was supported

10

What the customer really needed

# SDLC **without** Software Engineering

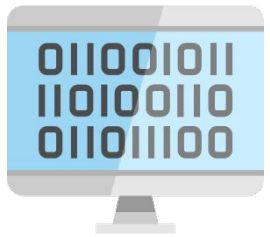
| Customer Requirement | Solution |
|---|---|
| • Have one trunk<br>• Have four legs<br>• Should carry load both passenger & cargo<br>• Black in color<br>• Should be herbivorous | • Have one trunk<br>• Have four legs<br>• Should carry load both passenger & cargo<br>• Black in color<br>• Should be herbivorous |
| | Our value added, also gives milk |

The software development **process** needs to be **engineered** to avoid the **communication gap** & to **meet the actual requirements** of customer within **stipulated budget** & **time**

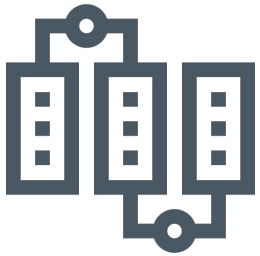
# What is Software?

## Software is

1) **Computer program** that when executed provide desired features, function & performance
2) **Data Structure** that enable programs to easily manipulate information
3) **Descriptive information** in both hard and soft copy that describes the operation and use of programs



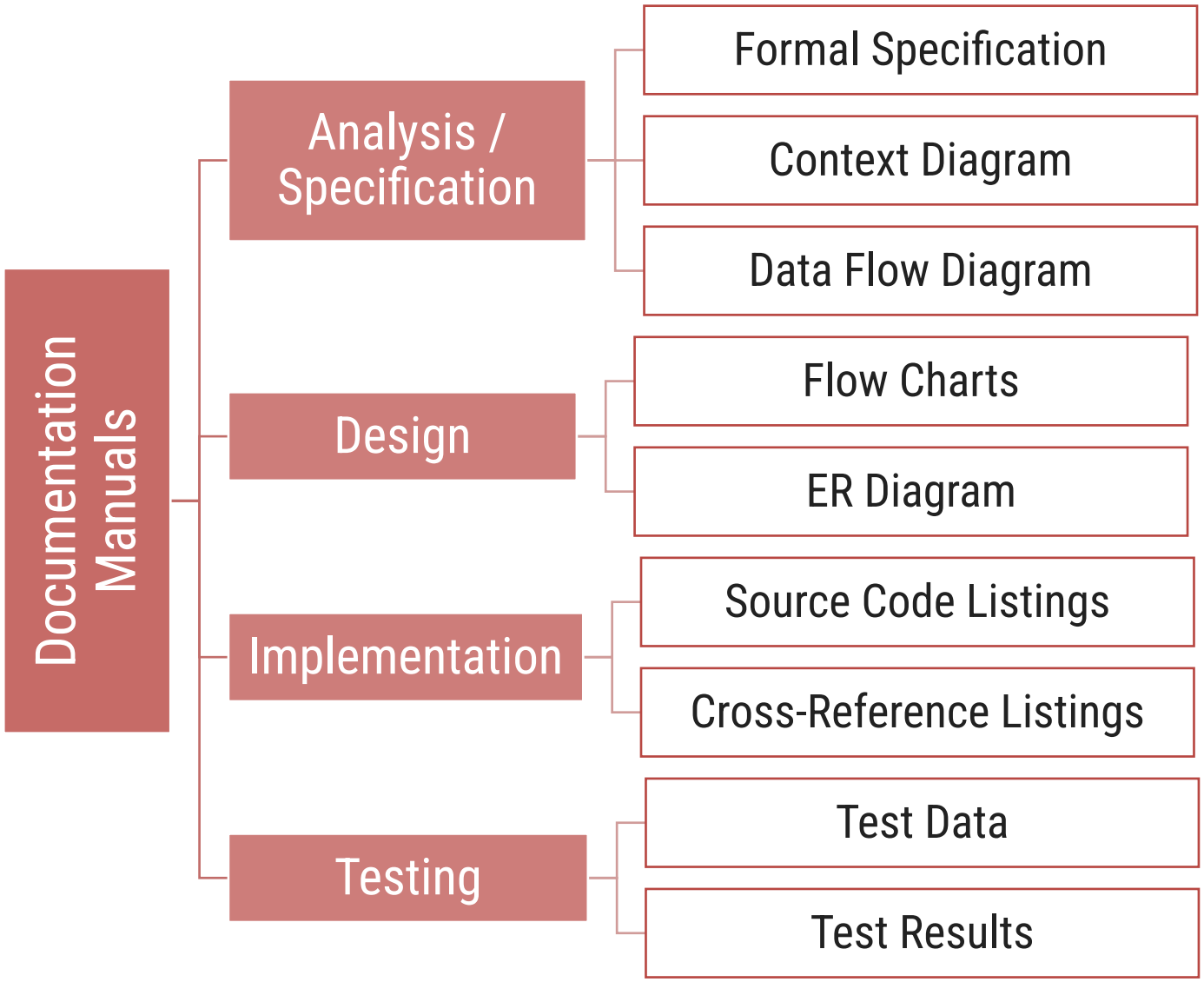
**Computer Program**

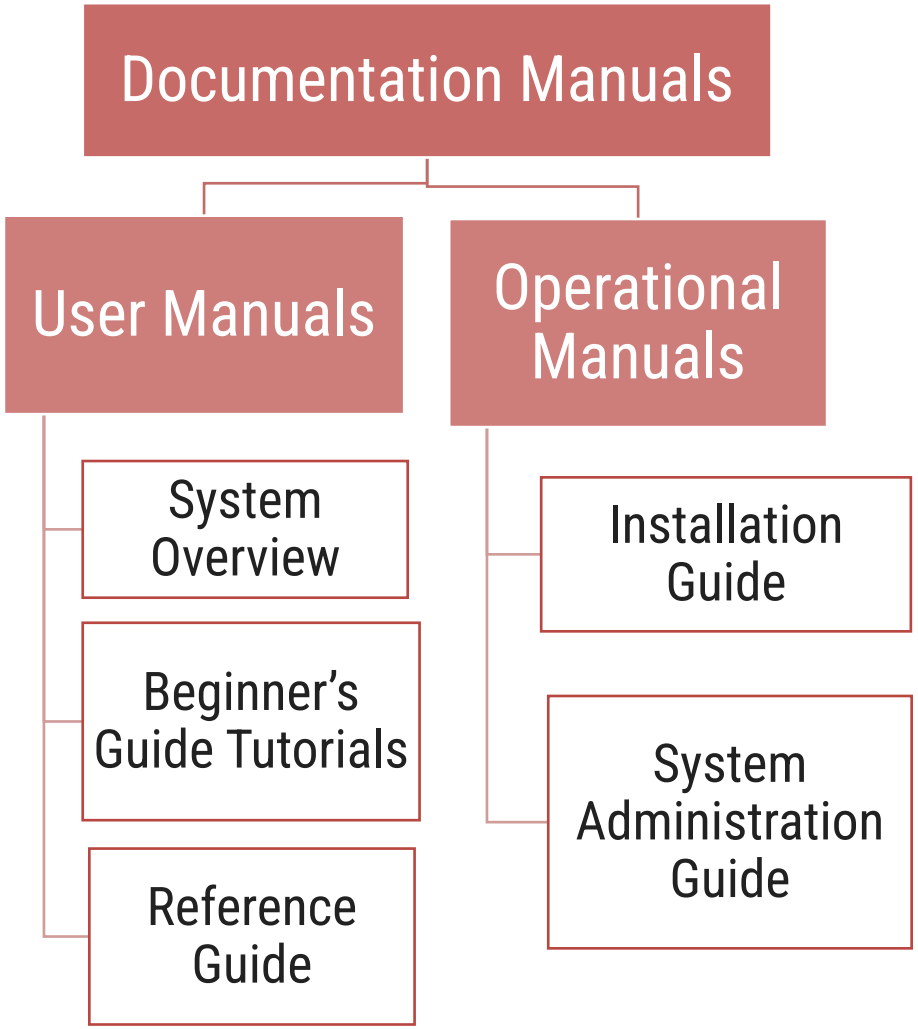


**Data Structure**

**Documents Soft & Hard**

# List of documentation & manuals



# Software Engineering

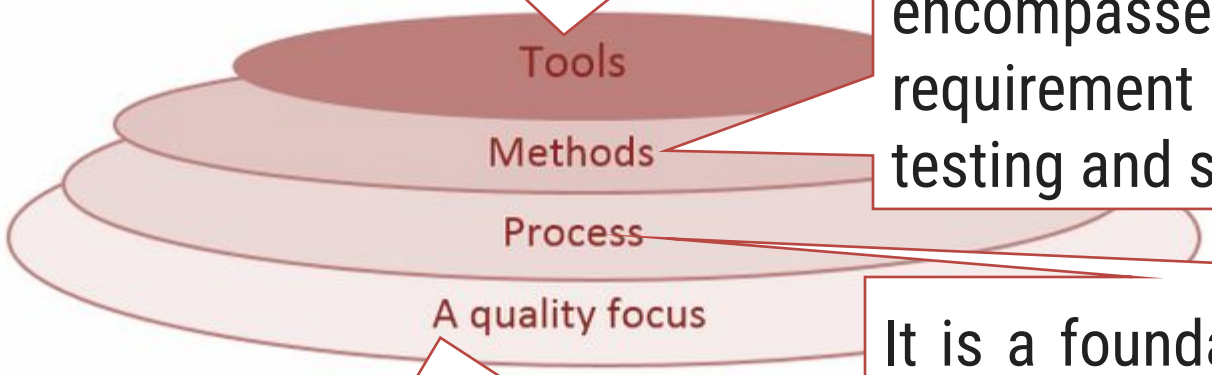
**Software engineering** is the establishment and use of **sound engineering principles** in order to obtain **economically software** that is **reliable and works** efficiently in **real machines**.

Software Engineering is the science and art of building (designing and writing programs) a software systems that are:

1) on **time**
2) on **budget**
3) with acceptable **performance**
4) with **correct operation**

# Software Engineering Layered Approach

Software Engineering Tools **allows automation of activities** which helps to perform systematic activities. A system for the support of software development, called **computer-aided software engineering** (CASE).
**Examples:** Testing Tools, Bug/Issue Tracking Tools etc…




It provides **technical how-to's** for building software, it encompasses **many tasks** including communication, requirement analysis, design modeling, program construction, testing and support

It is a foundation of Software Engineering, It is the **glue** the holds the **technology layers**, It **defines** a **framework activities**

Defines continuous **process improvement principles**

# Software Engineering Layered Approach Cont.

## Quality

- Main principle of Software Engineering is Quality Focus.
- An **engineering approach** must have a **focus on quality**.
- Total Quality Management **(TQM)**, **Six Sigma**, **ISO** 9001, ISO 9000-3, CAPABILITY MATURITY MODEL **(CMM)**, **CMMI** & similar approaches encourages a continuous process improvement culture

## Process Layer

- It is a foundation of Software Engineering, It is the glue the holds the technology layers together and enables logical and timely development of computer software.
- It **defines** a **framework** with activities for effective delivery of software engineering technology
- It establish the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.

# Software Engineering Layered Approach Cont.

## Method

- It provides **technical how-to's** for building software
- It **encompasses many tasks** including communication, requirement analysis, design modeling, program construction, testing and support

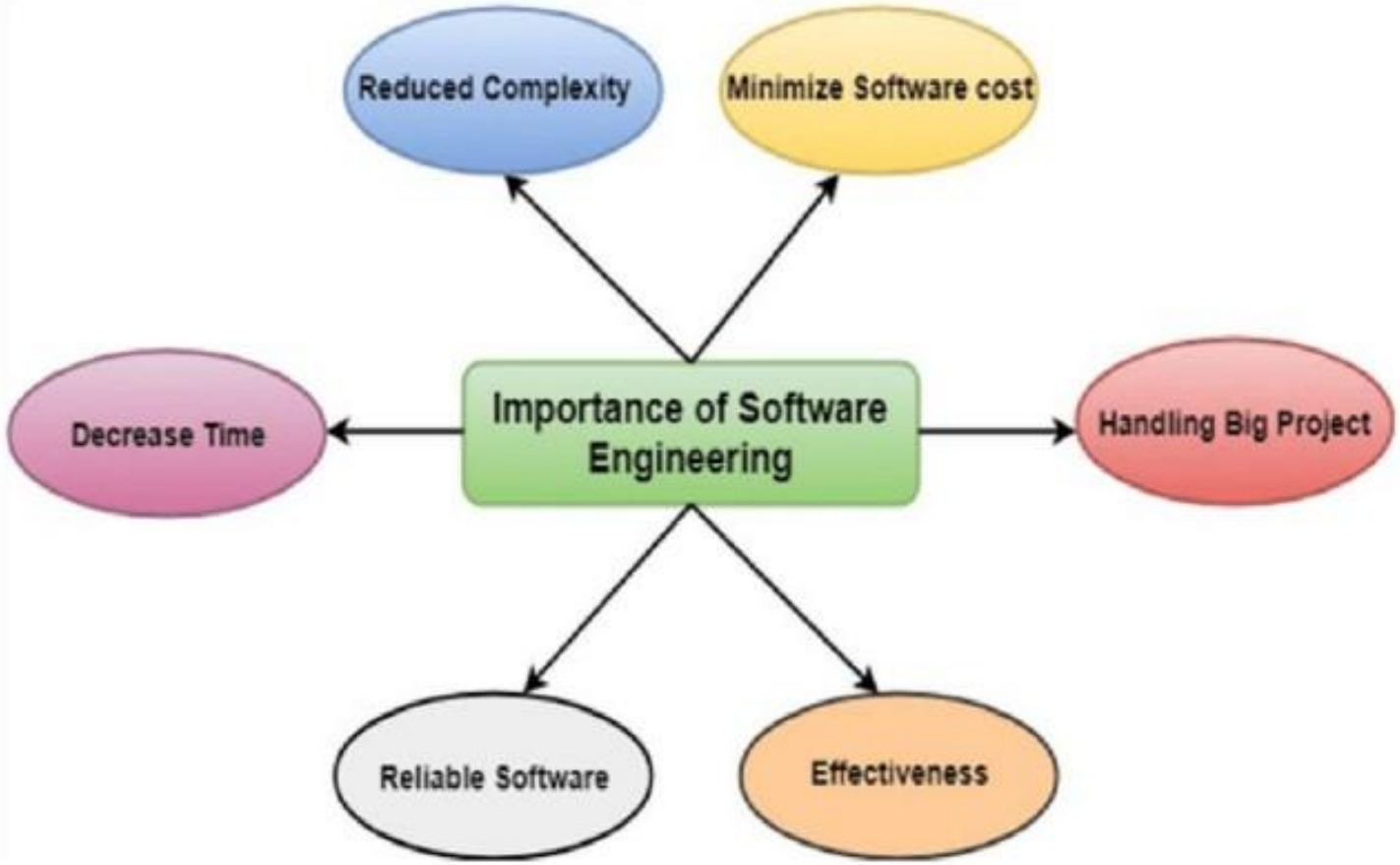
## Tools

- Software engineering tools provide automated or semiautomated support for the process and the methods
- Computer-aided software engineering (**CASE**) is the scientific application of a **set of tools** and **methods** to a software system which is meant to **result in high-quality, defect-free, and maintainable software products**.
- CASE tools automate many of the activities involved in various life cycle phases.

# The Evolving Role of Software

- Change in requirement with time
- Environment change
- Errors and bugs
- Security risks
- For having new functionality and features

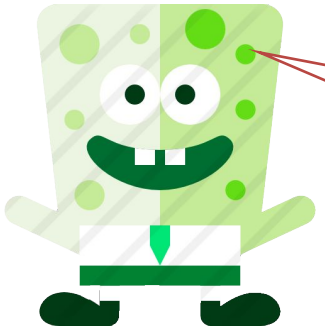
# Importance of Software Engineering



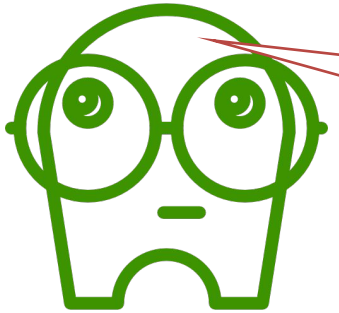
# Software Myths

Beliefs about software and the process used to build it.

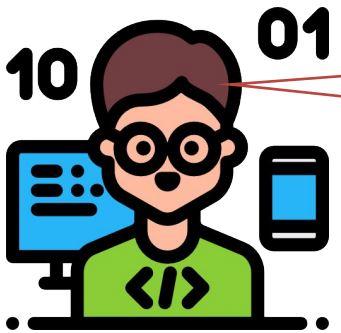
"**Misleading Attitudes that cause serious problem**" are myths.



Management Myths



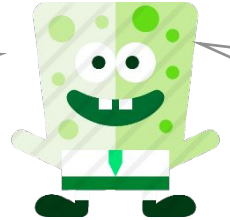
Customer Myths




Practitioner's (Developer) Myths

# Management myth - 1 & 2

We **have standards and procedures** to build a system, which is enough.



We have **the newest computers and development tools**.

**Reality**

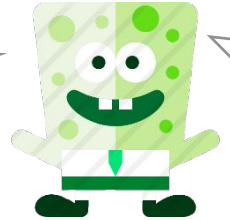
- Are software **practitioners aware** of standard's existence?
- Does it **reflect modern software engineering** practice?
- Is it **complete**?
- Is it streamlined to **improve time to delivery** while **still maintaining a focus on quality**?
- In many cases, the answer to all of these questions is "no."

**Reality**

- It **takes much more than the latest model** computers to do high-quality software development.
- **Computer-aided software engineering (CASE)** tools are more important than hardware.

# Management myth - 3 & 4

We **can add more programmers** and can catch up the schedule.



I **outsourced the development** activity, now I **can relax** and **can wait** for the **final** working **product**.

**Reality**

- Software **development is not a mechanistic process** like manufacturing.
- In the words of Fred Brooks : "**adding people to a late software project makes it later.**"
- **People** who were **working** must **spend time educating** the newcomers.
- People can be added but only **in a planned and well-coordinated** manner.

**Reality**

- If an **organization** does **not understand how to manage** and **control** software projects internally, it will invariably struggle when it outsources software projects.

# Customer myth - 1 & 2

A **general statement of objectives** (requirements) is **sufficient** to start a development.

**Requirement Changes** can be **easily accommodated** because software is very flexible.

**Reality**

- Comprehensive (**detailed**) **statements** of requirements is not always possible, an **ambiguous** (unclear) "**statement of objectives**" can lead to disaster.
- Unambiguous (clear) requirements can be gathered only through effective and continuous communication between customer and developer.

**Reality**

- It is true that software **requirements change**, but the **impact** of change **varies with the time** at which it is introduced.
- When requirements changes are requested early the cost impact is relatively small.

# Practitioner's (Developer) myth – 1 & 2

**Once** we **write** the **program**, our **job is done**.

I **can't** access **quality until** it is **running**.

**Reality**

- Experts say "**the sooner you begin 'writing code', the longer it will take you to get done.**"
- Industry data indicates that 60 to 80 % effort expended on software will be after it is delivered to the customer for the first time.

**Reality**

- One of the most effective software **quality assurance mechanisms** can be **applied from** the **beginning** of a project - **the technical review**.
- Software reviews are more effective "quality filter" than testing for finding software defects.

# Practitioner's (Developer) myth – 3 & 4

Working **program** is the **only deliverable** work **product**.

**Software engineering** is about **unnecessary** documentation.

**Reality**

- **A working program** is only one **part of** a **software configuration**.
- A variety of work products (e.g., **models, documents, plans**) provide a foundation for successful engineering and, more important, guidance for software support.

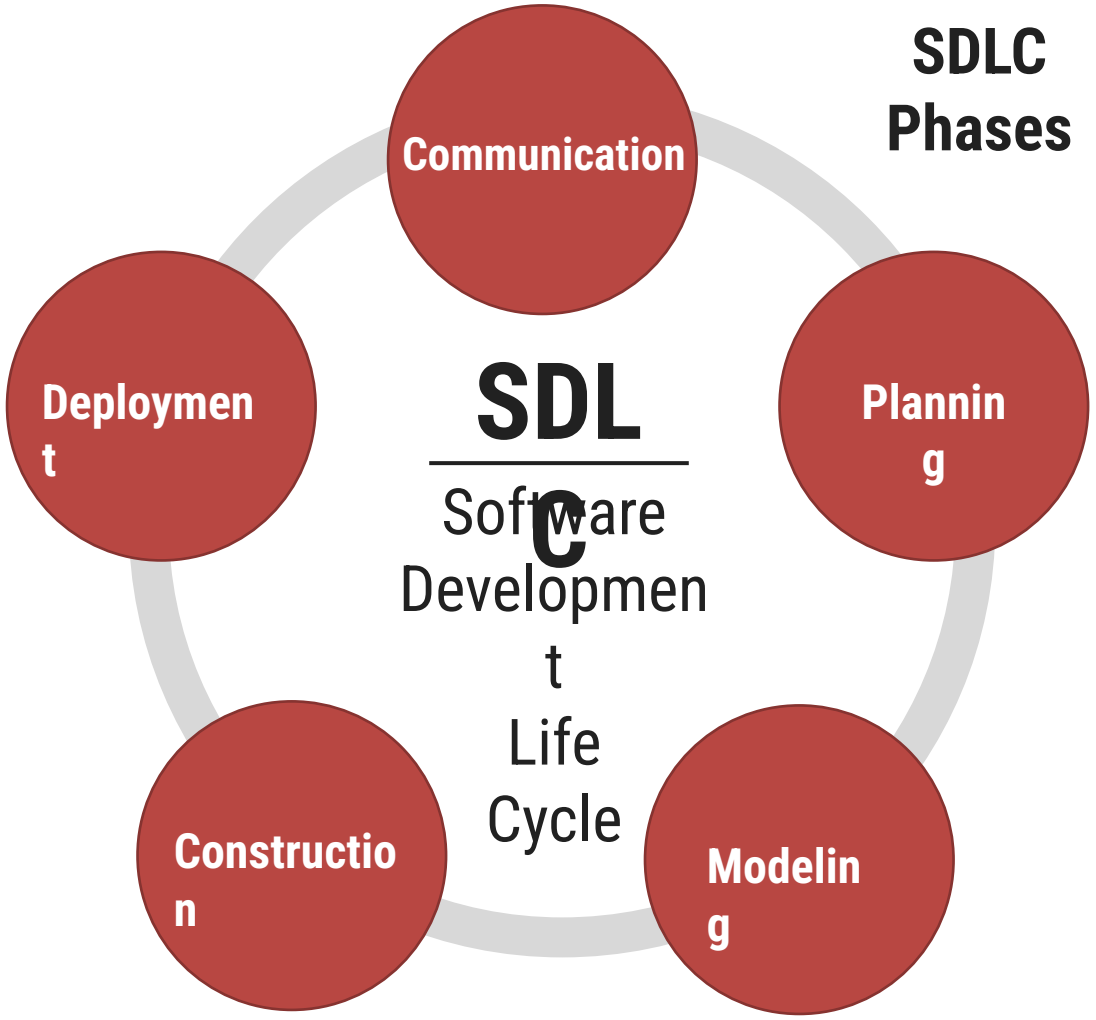
**Reality**

- Software engineering is not about creating documents. It is about **creating a quality product**.
- Better quality leads to reduced rework. And reduced rework results in faster delivery times.

# Software Process Models

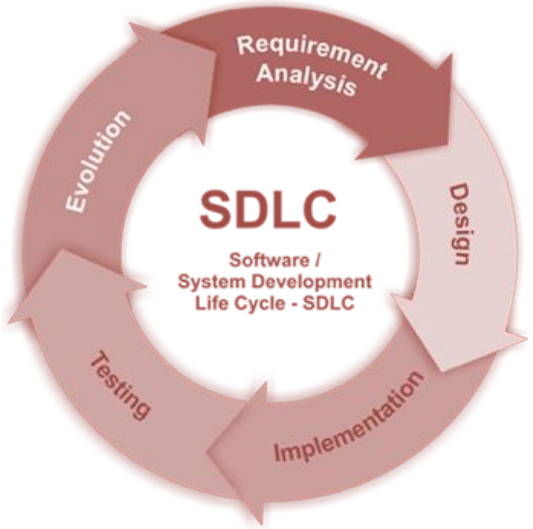
The **process model** is the abstract representation of process.

- Also known as **Software development life cycle (SDLC)** or Application development life cycle Models
- Process models **prescribe** a distinct set of **activities, actions, tasks and milestones (deliverables)** required to engineer high quality software.
- Process **models are not perfect,** but **provide roadmap** for software engineering work.
- Software models provide stability, control and organization to a process that if not managed can easily get out of control.
- Software process models are adapted (adjusted) to meet the needs of software engineers and managers for a specific project.



# Different Process Models

- Process model is selected based on different parameters
  - Type of the project & people
  - Complexity of the project
  - Size of team
  - Expertise of people in team
  - Working environment of team
  - Software delivery deadline




## Process Models

- Waterfall Model (Linear Sequential Model)
- Incremental Process Model
- Prototyping Model
- The Spiral Model
- Rapid Application Development Model
- V Model
- Agile Model

# Risk Management in Software Engineering

# Types of Risks

- Project Risks
- Technical Risks
- Business Risks
- External risks

# Thank You